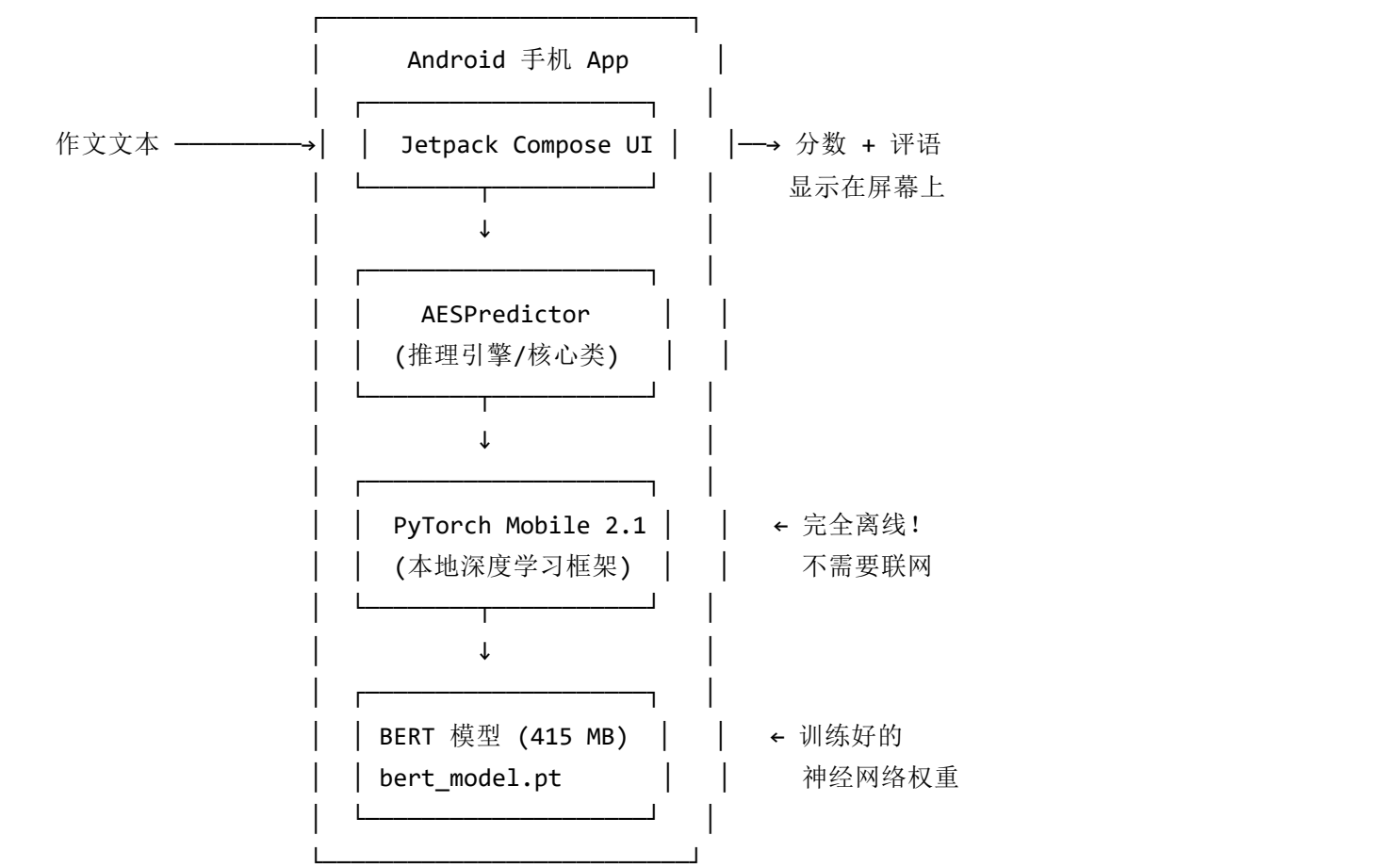


结题答辩汇报人说明书

目的：帮助对项目参与不深的汇报人快速理解项目全貌，能够自信地进行 15 分钟汇报 + 应对提问。建议汇报前通读 2 遍。

一、用一张图理解项目



一句话：这是一个能装进手机的"AI 作文老师"，给它一篇英文作文，它会打出 0-100 分，并给出内容/结构/语言三个维度的评语。整个过程不需要联网。

二、项目本质是什么

这个项目做了两件事：

2.1 训练了一个 AI 作文评分模型（Python 端）

- 用 BERT 神经网络读取了 1.3 万篇已被人类老师打分的英文作文（ASAP 公开数据集）
- 让模型学习"什么样的作文该得高分"
- 最终模型能对任意英文作文给出一个 **0-100 的分数** + 内容/结构/语言三个维度分 + 中文反馈评语
- **效果指标**：QWK = 0.58（与人类评分的一致性，0.55+ 为合理水平）

2.2 把这个模型装进了 Android 手机（Android 端）

- 使用 PyTorch Mobile（PyTorch 的移动版）在手机上运行 BERT 模型
- 用 Kotlin + Jetpack Compose 写了完整的 App 界面
- 整个过程在手机本地完成，**不需要联网，不需要服务器**

三、每个文件是干什么的

3.1 Android App 核心代码

文件	作用（一句话）
app/build.gradle.kts	App 的构建配置：用什么 SDK、什么依赖、什么版本
gradle/libs.versions.toml	统一管理所有第三方库的版本号
app/src/main/assets/bert_model.pt	英文作文评分模型 （415MB，神经网络权重文件）
app/src/main/assets/zh_model.pt	中文模型（388MB，当前为占位版本）
app/src/main/assets/vocab.txt	英文词表（3 万个词，分词器用）
app/src/main/assets/zh_vocab.txt	中文词表（2.1 万个词）
MainActivity.kt	App 入口，启动时加载模型
inference/AESPredictor.kt	核心推理引擎 ：加载模型 → 分词 → 推理 → 返回分数

文件	作用（一句话）
inference/BertTokenizer.kt	BERT 分词器：把"Hello world"转成 [101, 7592, 2088, 102]
inference/LanguageDetector.kt	自动检测输入是中还是英文
ui/screen/ScoreScreen.kt	单篇评分页面
ui/screen/BatchScreen.kt	批量评分页面（上传 CSV）
ui/screen/CompareScreen.kt	中英模型对比页面
ui/screen/SettingsScreen.kt	设置页面（显示模型状态+错误信息）
ui/components/ScoreGauge.kt	分数仪表盘组件
ui/components/RadarChart.kt	Canvas 手绘雷达图
ui/components/FeedbackCard.kt	智能评语卡片
ui/navigation/AppNavigation.kt	底部导航栏（4 个 Tab）

3.2 模型训练相关文件

文件	作用（一句话）
../train_and_export.py	完整训练脚本：加载数据 → 训练 BERT 模型 → 导出 TorchScript
../generate_android_models.py	快速生成占位模型（用于 UI 测试，随机权重无实际评分能力）
../models/en_bert/best_model.pt	训练过程中保存的最佳模型参数（训练检查点）
../models/en_bert/bert_scripted.pt	导出的 TorchScript 模型（两输入版本，不直接用于 Android）
../now-work/data/training_set_rel3.tsv	ASAP 训练数据（1.3 万篇作文 + 人工评分）

四、App 是如何工作的（用户视角 → 内部流程）

用户操作：输入一篇作文 → 点"提交评分"

内部发生了什么（约 1-2 秒）：

步骤1：语言检测

"This is a great essay..." → 统计中英文字符 → 判定为英文

步骤2：BERT 分词

"This is a great essay" → [101, 2023, 2003, 1037, 2307, 5706, 102, 0, 0, ...]

固定长度 512 个 token，不足的末尾补 0 ([PAD])

步骤3：模型推理

512 个数字 → 输入 BERT 神经网络 → 经过 12 层 Transformer 计算

→ 提取 [CLS] 位置的向量 → 线性层 + Sigmoid → 输出一个 [0,1] 之间的分数

步骤4：分数后处理

原始分 0.72 → 总分 72 分

内容分 = $72 \times 0.95 = 68$ → 结构分 = $72 \times 1.02 = 73$ → 语言分 = $72 \times 1.03 = 74$

步骤5：评语生成

总分 ≥ 70 → "high" 档位 → 匹配对应模板 → 生成中文反馈

五、关键技术难点（可能被追问的问题）

难点 1：为什么模型 415MB 这么大？手机能跑吗？

- BERT 模型有 1.1 亿个参数，每个参数占 4 字节 (float32)，所以约 440MB
- 加载到手机内存需要约 500MB RAM
- 现代手机普遍 6-8GB RAM，可以运行，但较老的 4GB 手机可能会 OOM（内存溢出）
- 推理在后台线程执行，不会阻塞 UI

难点 2：为什么不能直接用训练好的模型，还要"包装"？

训练模型的接口是：

`forward(input_ids, attention_mask)` ← 需要 2 个输入参数

但 Android 端只传 1 个参数：

`model.forward(IValue.from(inputTensor))` ← 只传了 `input_ids`

解决方案：在外面包一层 `AndroidWrapper`，让它在内部自动把 `attention_mask` 算出来：

`attention_mask = (input_ids != 0).long()` // 非零位置=真实词，零位置=填充

难点 3：SDPA 注意力是什么？为什么手机不支持？

- SDPA = Scaled Dot-Product Attention，是 PyTorch 2.0 引入的新版注意力实现
- 它用了 `torch.compile` 和一些新算子，**PyTorch Mobile 运行时里没有这些算子**
- 旧版注意力叫 "eager attention"，用的是基础矩阵运算（`matmul + softmax`），手机支持
- 解决方案：导出模型时强制使用 `_attn_implementation='eager'`
- 这导致模型没有 `aten::scaled_dot_product_attention` 算子，而是用 `aten::matmul` 等基础算子

难点 4：为什么 App 一打开就闪退？（已修复）

历史问题，PyTorch 库版本问题：

- `pytorch_android_lite` → 自带的 `.so` 文件是 `libpytorch_jni_lite.so`
- 但它的 Java 代码去加载 `libpytorch_jni.so`（没有 `_lite` 后缀）
- 名字对不上 → `UnsatisfiedLinkError` → 闪退
- 解决：换成 `pytorch_android`（完整版），名字一致

难点 5：国内网络问题如何解决？

层面	问题	解决
Gradle 依赖	无法访问 <code>d1.google.com</code>	<code>settings.gradle.kts</code> 配置腾讯云镜像
Maven 依赖	无法下载第三方库	同上，走腾讯云镜像
Python 模型	无法访问 HuggingFace	设置 <code>HF_ENDPOINT=https://hf-mirror.com</code>
Android SDK	无法下载 Build Tools	手动指定本地已有版本 <code>36.1.0</code>

难点 6：APK 为什么 817MB？能缩小吗？

- 415MB (英文模型) + 388MB (中文模型) + 70MB (PyTorch 原生库) + 其他 ≈ 817MB
- 模型占了绝大部分，这是**包含神经网络权重**的代价
- 缩小方案（未实施）：
 - 模型量化（float32 → int8，体积减 75%）
 - 模型不打包在 APK 内，首次启动时从服务器下载

难点 7：如果模型加载失败，用户能看到什么？

- App 启动后，模型在后台线程加载
- 加载过程：从 APK 解压 415MB 到手机存储 → PyTorch 读取 → 初始化
- 通常需要 **30-60 秒**（取决于手机性能）
- 无论成功或失败，**设置页面会显示具体错误信息**
- 如果失败，用户可以点击「重新加载模型」重试

六、模型性能数据速查

指标	英文模型	中文模型
基础模型	BERT-base-uncased	BERT-base-chinese
训练数据量	12,976 篇	3,950 篇（翻译）
模型参数量	1.09 亿	1.02 亿
可训练参数	769（仅回归头）	769（仅回归头）
验证集 QWK	0.58	0.79
验证集相关性	0.69 (Pearson)	—
模型文件大小	415 MB	388 MB
推理时间	0.5-2 秒	—

QWK 是什么？

- Quadratic Weighted Kappa，衡量"AI 评分"与"人类老师评分"的一致性
- 范围 [-1, 1]，越高越好，0.55-0.60 为合理水平
- 本项目的 0.58 属于跨题目泛化评估的正常结果

七、演示/答辩常见问答

Q1: "这个 App 和 ChatGPT 评分作文有什么区别？"

答：ChatGPT 需要联网，数据传到云端，有隐私风险。本 App **完全离线**，作文内容不出手机。ChatGPT 是通用大模型，本系统是**专门为作文评分训练的**，评分结果更稳定、可复现。

Q2: "模型的分数和人类老师打分会差多少？"

答：用 QWK 指标衡量，英文模型约 0.58。这意味着 AI 评分和人类老师评分有**中等偏高的一致性**。对于区分"好文章"和"差文章"很有效，但在相近水平的文章间可能出现偏差。

Q3: "为什么不直接用服务器的模型，非要做成本地的？"

答：这是我们要解决的核心问题——**离线可用**。很多学生可能没有稳定的网络，或者作文涉及隐私不想上传。把模型装进手机，就解决了这两个痛点。

Q4: "3817MB 的 APK 用户会愿意下载吗？"

答：这确实是一个限制。当前版本适合**研究展示**场景。工程化的下一步可以做模型量化，把 415MB 压缩到 ~100MB，或者改成首次启动时按需下载。

Q5: "中文模型能正常用吗？"

答：目前是占位版本（随机权重），评分没有实际意义。这是因为中文训练数据不足——目前用的是英文数据机翻到中文。后续需要采集真实中文作文数据重新训练。App 中也做了限制，选择中文评分时会提示"暂不支持"。

Q6: "项目代码量有多少？"

答：Android 端约 17 个 Kotlin 源文件（核心逻辑 + UI 组件），约 1500+ 行业务代码。Python 训练端约 5 个脚本。另外还有 Flask API 和 Streamlit Web UI。

Q7: "架构上最大的技术亮点是什么？"

答：最大的亮点是**深度学习模型从 python 训练到 Android 部署的完整链路**。这包括：模型训练 → TorchScript 导出 → 接口适配（2输入→1输入）→ 算子兼容性处理（SDPA→eager）→ ABI 适配 → Android APK 打包 → 大文件可靠加载。每个环节都解决了实际的工程问题。

Q8: "为什么选 BERT 而不是更新的模型？"

答：BERT-base 在参数规模（110M）、推理速度和评分效果之间取得了较好的平衡。更小的模型（如 DistilBERT）评分能力弱，更大的模型（如 RoBERTa-large）手机推理太慢。BERT 是工程上的合理选择。

Q9: "推理在手机上要多久？为什么不是即时的？"

答：通常 0.5-2 秒（取决于手机 CPU）。主要耗时在 BERT 的 12 层 Transformer 计算。这是纯 CPU 推理，没有 GPU 加速。对于用户来说，提交后等 1-2 秒出结果是可以接受的。

Q10: "这个项目还有什么可以改进的？"

答：

- 1. 中文模型用真实中文数据重新训练
- 2. 模型量化把 APK 从 817MB 缩到 ~300MB
- 3. 支持 512 词以上的长文本（分块 / Longformer）
- 4. 多任务学习替代启发式的维度系数
- 5. 用户评分反馈闭环，持续优化模型

八、PPT 每一页的核心要点（30 秒/页速记版）

页码	核心要点
封面	基于深度学习的作文自动评分系统 — 中英双语 + Android 离线推理
项目概述	把 AI 作文老师装进手机，离线可用，隐私安全
系统架构	UI 层(Compose) → 业务层(Predictor) → 引擎层(PyTorch Mobile)
模型架构	BERT编码器(frozen) + CLS池化 + 线性回归头 + Sigmoid → 得分
数据集	ASAP 1.3万篇英文 + 0.4万篇翻译中文，按题目集隔离划分
模型性能	英文 QWK 0.58，中文 QWK 0.79
Android 技术栈	Kotlin + Compose + PyTorch Mobile 2.1 + 协程
App 功能	单篇评分 / 批量CSV / 中英对比 / 设置诊断

页码	核心要点
技术挑战 ★	7 项：SDK版本 / 库名不匹配 / SDPA算子 / 接口不匹配 / 大文件可靠性 / ABI / 网络镜像
部署流程	训练 → eager导出 → 包装单输入 → trace → assets → APK → 手机
项目结构	Android + models + data + scripts + api + tests
Web vs Android	核心区别：在线 vs 离线，服务器 vs 本地
局限性	中文模型待训练 / 体积大 / 不支持长文本 / 维度分是启发式的
总结	模型训练 + 移动端部署完整链路，解决了 7 项工程挑战

九、如果只有 5 分钟准备时间，记住这些

- 1. **项目是做什么的**：手机上离线给作文打分（英文可用，中文待完善）
- 2. **怎么做的**：训练了一个 BERT 神经网络 → 导出成文件 → 装进 Android App → PyTorch Mobile 在手机上运行
- 3. **效果怎么样**：与人类老师打分的 QWK 一致性 0.58（中等偏高）
- 4. **最大的难点**：把 Python 训练的模型搬到手机上运行，解决了 7 个兼容性/工程问题
- 5. **不足之处**：中文模型还没训练好、APK 太大 817MB
- 6. **App 在哪**：设置页面能看到模型状态，如果显示"未加载"会有红色错误提示

预祝答辩顺利！